

Unidade: Seguridade no uso de Docker

0. Influencia do uso de contedores na seguridade

Os contedores achegan vantaxes significativas dende o punto de vista da seguridade, sempre que se configuren correctamente. Non son unha capa máxica de protección, pero permiten illar procesos, controlar recursos e reducir riscos.

0.1. Illamento de procesos e recursos

Cada contedor executa os seus procesos, usuarios, rede e sistema de ficheiros nun espazo de nomes separado. Isto impide que un contedor poida acceder a outros ou ao host directamente.

Beneficio: se unha aplicación é comprometida, o atacante queda “encerrado” dentro do contedor.

0.2. Entornos reproducibles e controlados

As imaxes son inmutables e versionadas. Permiten repetir despregamentos de forma idéntica e auditar configuracións. Isto mellora a **integridade** e a **trazabilidade** das aplicacións.

0.3. Redución da superficie de ataque

As imaxes minimalistas conteñen só o necesario para executar a aplicación, eliminando compoñentes e servizos innecesarios.

0.4. Configuración declarativa

A seguridade é parte da infraestrutura como código. A configuración está en ficheiros ([Dockerfile](#), [compose.yml](#), manifests Kubernetes), facilitando auditorías e revisións automáticas.

0.5. Integración con ferramentas de seguridade

O ecosistema Docker/Kubernetes integra ferramentas para análise, monitorización e control:

- Trivy, Grype: escaneo de vulnerabilidades.
- Falco, Sysdig: detección de comportamentos anómalos.
- Vault, Sealed Secrets: xestión de secrets.

0.6. Limitacións e riscos do uso de contedores

Aínda que os contedores proporcionan un illamento lóxico e eficaz, non son unha fronteira de seguridade absoluta. O illamento depende do kernel do sistema operativo e da configuración do daemon de Docker, polo que existen riscos que deben considerarse.

Container escape

Un *container escape* ocorre cando un proceso dentro dun contedor consegue executar **código no host** ou acceder a outros contedores. Isto pode deberse a:

- Vulnerabilidades no kernel de Linux ou nas interfaces de *namespaces* e *cgroups*.
- Montaxes perigosas, como `-v /var/run/docker.sock:/var/run/docker.sock` que permitiría controlar dende o contedor o daemon de Docker do host.
- Contedores executados con opcións privilexiadas (`--privileged`) ou capacidades excesivas (`--cap-add=ALL`).

Escalada de privilexios

Docker require permisos de administración para xestionar contedores. Se un atacante obtén acceso ao daemon (`dockerd`), **ten control total sobre o host**. Por iso, non se debe engadir usuarios comúns ao grupo `docker` nin executar contedores como `root` cando non é necesario.

Imaxes e dependencias non confiables

Executar imaxes descargadas de repositorios públicos sen verificación pode introducir *malware*, *backdoors* ou vulnerabilidades. É esencial empregar imaxes oficiais, asinadas ou revisadas, e realizar escaneos periódicos con ferramentas como **Trivy** ou **Grype**.

Configuracións inseguras

Outros riscos comúns inclúen:

- Exposición de portos innecesarios.
- Montaxe de volumes con permisos de escritura sobre o host.
- Falta de control no acceso a secrets e variables de contorno sensibles.
- Falta de actualización nas imaxes base.

Resumo: os contedores facilitan aplicar o principio de *mínimo privilexio*, automatizar revisións e manter un entorno máis controlado e reproducible.

Obxectivos da sesión

- Comprender os riscos máis comúns no uso de contedores.
- Aprender a analizar e mellorar a seguridade das imaxes Docker.
- Xestionar secrets e variables sensibles de forma segura.
- Aplicar boas prácticas de execución e integración DevSecOps.

1. Introducción: riscos e superficie de ataque

Que é DevSecOps?

DevSecOps é a evolución natural de DevOps: engadir a seguridade dende o inicio do ciclo de vida. Non é algo que se engade ao final, senón unha parte do proceso de desenvolvemento e despregamento.

"Shift Left Security": significa que a seguridade se incorpora canto antes, non cando o sistema xa está en produción.

Riscos comúns en contedores

Risco	Exemplo	Consecuencia
Imaxes vulnerables	Usar <code>ubuntu:latest</code> sen escanear	Exploits coñecidos
Secrets en Dockerfile	<code>ENV TOKEN=1234abcd</code>	Fuga de credenciais
Usuario root	<code>USER root</code>	Escalada de privilexios
Dependencias desactualizadas	<code>apt install</code> sen fixar versións	Paquetes inseguros
Red exposición excesiva	<code>EXPOSE 0.0.0.0:80</code>	Acceso non controlado

Exemplo de Dockerfile inseguro

```
FROM ubuntu:latest
RUN apt update && apt install -y curl
ADD config.json /root/
ENV TOKEN=1234567890abcdef
USER root
```

Actividade:

1. Identifica 3 riscos neste Dockerfile.
2. Propón unha corrección para cada un.

2. Análise e scanning de imaxes

Ferramentas útiles

Ferramenta	Tipo	Características
<code>trivy</code>	Open Source	Escanea imaxes, ficheiros e dependencias
<code>grype</code>	Anchore	Alternativa lixeira e precisa
<code>snyk</code>	SaaS	Similar ao antigo <code>docker scan</code> , require conta

Nota: O comando `docker scan` foi retirado das versións recentes do Docker CLI. Recoméndase empregar `trivy`, que é libre e máis completa.

Exemplo práctico

```
# Escaneo rápido
trivy image nginx:latest

# Alternativa
grype nginx:latest
```

Actividade:

1. Escanea unha imaxe base (por exemplo `python:3.11-slim`).
2. Identifica vulnerabilidades críticas (`CRITICAL` ou `HIGH`).
3. Comenta posibles solucións.

3. Boas prácticas na construción de imaxes

Práctica	Descrición	Exemplo
Usar imaxes base minimalistas	Menos superficie de ataque	<code>FROM debian:bookworm-slim</code>
Fixar versións concretas	Evita cambios inesperados	<code>FROM python:3.11-slim</code>
Non usar <code>ADD</code> sen necesidade	Prefire <code>COPY</code>	<code>COPY app.py /app/</code>
Non executar como root	Crea usuario non privilexiado	<code>USER appuser</code>
Limpar caches e listas	Reduce tamaño e riscos	<code>rm -rf /var/lib/apt/lists/*</code>
Asinar imaxes	Garante orixe e integridade	<code>cosign sign myimage:latest</code>

Actividade: Reescribe o Dockerfile inseguro anterior aplicando boas prácticas.

4. Manexo seguro de secrets e variables sensibles

Boas prácticas

- Nunca gardar secrets no Dockerfile nin en variables fixas de imaxe.
- Non subir ficheiros `.env` nin secrets a Git.
- Usar ficheiros externos, variables de entorno ou servizos de secrets.
- En `docker-compose`, empregar o campo `env_file` ou a sección `secrets`.

Alternativas

Método	Descrición	Exemplo
<code>.env</code> local	Ficheiro con variables de entorno cargado externamente	<code>docker run --env-file .env myapp</code> ou <code>env_file: .env</code> en Compose
<code>secrets</code> en Compose	Gardar secrets en ficheiros locais e montalos no contedor	<code>secrets:</code> en <code>docker-compose.yml</code>
Vault / AWS Secrets / Azure Key Vault	Xestión centralizada de credenciais	Acceso mediante API, non incluídas na imaxe

Exemplo práctico con `.env`

Ficheiro `.env`:

```
DB_USER=admin
DB_PASS=SuperSecret123
```

Execución directa:

```
docker run --env-file .env myapp
```

Versión con `docker-compose.yml`:

```
version: "3.9"
services:
  app:
    image: myapp:latest
    env_file:
      - .env
    environment:
      - ENV=production
```

Exemplo práctico con `secrets` en Docker Compose

1. Crear o secret localmente:

```
echo "SuperSecret123" > ./db_pass.txt
```

2. Ficheiro `docker-compose.yml`:

```
version: "3.9"
services:
  app:
    image: myapp:secure
    secrets:
      - db_pass
secrets:
  db_pass:
    file: ./db_pass.txt
```

3. Execución:

```
docker compose up -d
```

O secret estará dispoñible no contedor en `/run/secrets/db_pass`.

5. Seguridade en execución (runtime)

Parámetros seguros en Docker CLI

Opción	Propósito	Exemplo
<code>--user</code>	Evita procesos como root	<code>--user 1000:1000</code>
<code>--read-only</code>	Sistema de ficheiros só lectura	<code>--read-only</code>
<code>--cap-drop=ALL</code>	Elimina capacidades Linux extra	<code>--cap-drop=ALL</code>
<code>--security-opt no-new-privileges:true</code>	Bloquea escaladas	<code>--security-opt no-new-privileges:true</code>
<code>--memory, --cpus</code>	Limita recursos	<code>--memory=512m --cpus=0.5</code>

Exemplo CLI:

```
docker run -d --read-only --cap-drop=ALL --user 1000:1000 --memory=512m  
--cpus=1 myimage:secure
```

Versión con `docker-compose.yml`

```
version: "3.9"  
services:  
  web:  
    image: myimage:secure  
    read_only: true  
    user: "1000:1000"  
    cap_drop:  
      - ALL  
    security_opt:  
      - no-new-privileges:true  
    deploy:  
      resources:  
        limits:  
          cpus: "1.0"  
          memory: 512M
```

Explicación:

- `read_only` fai o FS do contedor de só lectura.
- `cap_drop` elimina capacidades do kernel.
- `user` define o usuario interno do contedor.
- `security_opt` aplica restricións adicionais.
- `deploy.resources.limits` controla uso de CPU e memoria.

6. Integración DevSecOps

Pipeline básico de seguridad con Docker Compose

Pódesse automatizar a seguridade combinando construción segura e escaneo de vulnerabilidades.

Exemplo de `docker-compose.yml` para análise:

```
version: "3.9"
services:
  app:
    build: .
    image: myapp:latest
    env_file:
      - .env
    read_only: true
    user: "1000:1000"
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true

  scanner:
    image: aquasec/trivy:latest
    command: image myapp:latest
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
```

Execución:

```
docker compose build
docker compose run scanner
```

Se `trivy` detecta vulnerabilidades críticas, a imaxe non debería despregarse.

Integración nun pipeline CI/CD (exemplo simplificado)

```
stages:
  - build
  - scan
  - deploy

build:
  stage: build
  script:
    - docker compose build

scan:
```

```
stage: scan
script:
  - docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy
image myapp:latest

deploy:
  stage: deploy
  script:
    - docker compose up -d
  when: on_success
```

Kubernetes

Kubernetes amplía o modelo de contedores a nivel de orquestración, e con el tamén aparecen novos retos de seguridade. Veranse as principais prácticas para manter un clúster seguro e controlado.

7.1. Autenticación e autorización

Kubernetes emprega dous mecanismos fundamentais:

Compoñente	Función	Exemplos
Autenticación (AuthN)	Verifica quen accede ao clúster	Certificados, tokens, integración con OIDC
Autorización (AuthZ)	Controla que pode facer ese usuario	RBAC (Roles e RoleBindings)

Recomendacións:

- Crear roles específicos por aplicación ou equipo.
- Empregar **Role** e **RoleBinding** en lugar de **ClusterRole** cando sexa posible.
- Desactivar a conta **default** dos namespaces se non é necesaria.

Exemplo básico de Role:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: produccion
  name: lector-pods
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
```

7.2. Seguridade dos contedores

Kubernetes executa contedores a través de **Pods**, que herdán restricións do manifesto e do runtime (Docker, containerd...).

Boas prácticas:

- Engadir `securityContext` para definir restricións.
- Executar como usuario non root.
- Evitar montaxes de volumes do host.

Exemplo:

```
apiVersion: v1
kind: Pod
metadata:
  name: app-segura
spec:
  containers:
  - name: app
    image: nginx:alpine
    securityContext:
      runAsUser: 1000
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
```

7.3. Políticas de seguridade (PSA)

As antigas **PodSecurityPolicies (PSP)** foron substituídas por **Pod Security Admission (PSA)**, que establece niveis de restrición por *namespace*:

Nivel	Descrición
Privileged	Sen restricións (uso administrativo)
Baseline	Regras básicas de seguridade (por defecto)
Restricted	Máximo nivel de illamento (sen root, FS só lectura, etc.)

Exemplo de configuración restrinxida:

```
apiVersion: pod-security.admission.config.k8s.io/v1beta1
kind: PodSecurityConfiguration
defaults:
  enforce: "restricted"
```

7.4. Control de rede

Por defecto, todos os Pods poden comunicarse entre si. As **NetworkPolicies** permiten restrinxir isto.

Exemplo: permitir só tráfico HTTP dentro dun namespace

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: permitir-http
  namespace: web
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
    - Ingress
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: backend
      ports:
        - protocol: TCP
          port: 80
```

7.5. Seguridade do plano de control e API

Boas prácticas clave:

- Desactivar accesos anónimos (`--anonymous-auth=false`).
- Habilitar auditoría de eventos (`audit-log-path`).
- Restringir acceso á API Server mediante VPN, firewall ou IP whitelist.
- Manter actualizadas as imaxes dos compoñentes (kube-apiserver, etcd, kubelet...).

7.6. Ferramentas de auditoría e control

Ferramenta	Uso	Descrición
kubectl-who-can	Auditoría de permisos	Lista quen pode executar accións sobre un recurso
kube-bench	Cumprimento CIS Benchmark	Analiza configuracións inseguras do clúster
kube-hunter	Escaneo de vulnerabilidades	Busca portas abertas e riscos de rede
Falco	Detección runtime	Monitoriza comportamentos sospeitosos en contedores

7.7. Resumo das boas prácticas

Área	Recomendación principal
Autenticación	Usar RBAC e eliminar accesos anónimos

Área	Recomendación principal
Contedores	Usuario non root e FS de só lectura
Políticas	Nivel "restricted" en PSA
Rede	Definir NetworkPolicies
Monitorización	Empregar Falco, kube-bench, logs de auditoría
Actualizacións	Manter clúster e imaxes actualizados

8. Exemplo

Imos tomar como base o exercicio 4 do laboratorio de kubernetes.

04-web-pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: web-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

04-web-deployment-pvc.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
          volumeMounts:
```

```

      - name: web-data
        mountPath: /usr/share/nginx/html
volumes:
  - name: web-data
    persistentVolumeClaim:
      claimName: web-pvc

```

04-writer-pod.yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: writer
spec:
  restartPolicy: Never
  containers:
    - name: writer
      image: alpine:3.20
      command: ["/bin/sh", "-c"]
      args:
        - |
          echo '<!doctype html><html><head><meta charset="utf-8">
<title>PVC</title></head>
          <body><h1>Contido persistente en PVC</h1><p>Este index.html persiste
aínda que os Pods se borren.</p></body></html>' > /data/index.html
          ; sleep 2
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: web-pvc

```

Imos facer o seguinte:

1. Engadir un bloque `securityContext` adecuado.
2. Definir unha `NetworkPolicy` que limite o tráfico só aos pods do mesmo namespace.
3. Explicar brevemente que nivel de seguridade aplicarías (baseline ou restricted) e por que.

web-deployment-pvc.yaml (endurecido)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2

```

```

selector:
  matchLabels:
    app: web
template:
  metadata:
    labels:
      app: web
  spec:
    securityContext:
      runAsNonRoot: true
      runAsUser: 101          # usuario 'nginx' en alpine adoita ser 101
      runAsGroup: 101
      fsGroup: 101
      seccompProfile:
        type: RuntimeDefault
    containers:
      - name: nginx
        image: nginx:1.25-alpine
        ports:
          - containerPort: 80
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities:
            drop: ["ALL"]
            # Se mantés o porto 80 con non-root: engadir 'NET_BIND_SERVICE' aquí
            # add: ["NET_BIND_SERVICE"]
        volumeMounts:
          - name: web-data
            mountPath: /usr/share/nginx/html
            readOnly: true
    volumes:
      - name: web-data
        persistentVolumeClaim:
          claimName: web-pvc

```

writer-pod.yaml (endurecido)

```

apiVersion: v1
kind: Pod
metadata:
  name: writer
spec:
  restartPolicy: Never
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 1000
    fsGroup: 1000
    seccompProfile:
      type: RuntimeDefault

```

```

containers:
  - name: writer
    image: alpine:3.20
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
    command: ["/bin/sh", "-c"]
    args:
      - |
        echo '<!doctype html><html><head><meta charset="utf-8">
        <title>PVC</title></head>
        <body><h1>Contido persistente en PVC</h1><p>Este index.html persiste
        aínda que os Pods se borren.</p></body></html>' > /data/index.html
        ; sleep 2
    volumeMounts:
      - name: data
        mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: web-pvc

```

(O ficheiro **web-pvc.yaml** non require cambios.)

NetworkPolicy: só tráfico dentro do mesmo *namespace*

Esta política limita tanto o **ingress** coma o **egress** para que só poidan comunicarse *pods* do mesmo *namespace*.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-only-same-namespace
spec:
  podSelector: {} # afecta a todos os pods do namespace
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector: {} # calquera pod do MESMO namespace
  egress:
    - to:
        - podSelector: {} # só a pods do MESMO namespace

```

💡 Nota: isto tamén bloquea o acceso a DNS e Internet. Se se precisa DNS, hai que engadir unha excepción para o servizo **kube-dns** no *namespace* **kube-system**.

Nivel de seguridade: *baseline* vs *restricted*

- **Restricted (PSS):** require *non-root*, `seccomp=RuntimeDefault`, sen *privilege escalation*, FS raíz *read-only* e sen capabilities adicionais.
- **Baseline (PSS):** permite excepcións prácticas, como `NET_BIND_SERVICE` para escoitar en portos <1024.

Nivel aplicado: *Baseline*, xa que Nginx escoita no porto **80** como non-root e precisa esa capability.

Se se cambia o porto a **8080**, pódese eliminar `NET_BIND_SERVICE` e cumprir o perfil **Restricted** sen outros cambios.

Para cumprir o perfil *restricted*, modifica Nginx para escoitar no porto 8080:

- engade un `ConfigMap` co `nginx.conf` personalizado con `listen 8080;`, ou
- arrinca Nginx con `-g 'daemon off;'` e monta un `conf.d/default.conf` propio.